Sketchpad: certified exact arithmetic with a small vision-language model

Mostafa 'Neo' Mohsenvand
MIT Media Lab

31 October 2026

Abstract

Language models remain unreliable on long arithmetic, and tool use moves the computation outside the network. We present Sketchpad, a method in which a 0.8-billion-parameter vision-language model, Qwen3.5-0.8B with a low-rank adapter, executes arithmetic procedures on a rendered sheet that serves as its only working memory. Each forward pass reads a three-cell view of the sheet and emits one step: a digit, a carry or borrow, and the next operation; a fixed runtime writes the outputs and moves the view. Because each step depends only on its view, correctness on the finite set of views is equivalent to correctness on all inputs of any length. We prove this, together with its necessity, composition and robustness to numerical perturbation, in Lean, and certify all 2,446 views on the deployment device, a laptop GPU running the model in a web browser. The certified model performs signed addition and subtraction, long multiplication and division, square roots and greatest common divisors at arbitrary length. It solved 200 of 200 pre-registered problems in the browser, and 220 of 220 sums of up to 1,000 digits and products of up to 20 digits, on which the strongest of ten other models, GPT-6.1 Sol, solved 217 and the strongest open-weight model, DeepSeek-V4-Pro, 199. Built on these procedures, a symbolic runtime solves, expands and factors polynomials with every number computed by the model. A multiplicative gate on the adapter recovers the base model exactly.

1 Introduction

Multi-step exact computation remains a weakness of language models. Chain-of-thought prompting[1,2] and changes to positional encoding and data format[3–9] extend the lengths at which transformers add and multiply correctly, but in each case accuracy is estimated on samples and carries no guarantee for unseen inputs. Frontier models are now accurate on most long-arithmetic problems: in our pre-registered comparison, GPT-6.1 Sol answered 217 of 220 correctly (Fig. 1c). Its three errors, on sums of 300 and 1,000 digits, are not distinguishable from correct answers. The common alternative, tool use[10–14], is accurate but places the computation outside the network.

We take a third route: the network performs the computation, and its correctness is verified exhaustively. In Sketchpad, a runtime lays out a problem on a rendered sheet and presents the model with one view, a column of three cells, at a time. The model reads the view through its vision encoder and, in a single forward pass, outputs the step's results and the next operation. The runtime writes the outputs to the sheet and moves the view. No state passes between steps except the sheet (Fig. 1a).

Because every step is a function of its view and the views form a finite set, the model can be verified by enumeration. Exactness on all 2,446 views is equivalent to exactness on every run of every length (Theorem 1). This observation is classical: it underlies Moore's analysis of experiments on finite automata[15], the verification of learned recursive programs by Cai et al.[16] and the certificate for universal computation by language-model decoding of Lewandowski et al.[17]. Our contribution is a system in which the hypothesis holds by construction for a pretrained model, together with a certificate measured on the deployment device. Specifically:

1. a vision-language model of 0.8 billion parameters with a LoRA adapter[18] that executes one procedural step per forward pass (Section 2);
2. formal statements, checked in Lean, of exactness on views, its necessity, composition of certified procedures and robustness to numerical perturbation, and a certificate of all 2,446 views on the target device (Section 3);
3. a calculator for signed integer arithmetic, long division, square roots and greatest common divisors at arbitrary length (Section 4);

4. polynomial algebra in which every number is produced by the certified procedures (Section 5);
5. a gate under which the adapted model is identical to the base model (Section 6).

2 The sheet machine

Model. Qwen3.5-0.8B[19] combines a vision encoder with a 24-layer language model in which six layers use full attention and eighteen use Gated DeltaNet[20]. We train a rank-16 LoRA adapter on 150 projection matrices of the language model (10.2 million parameters, 1.2% of the model); all base weights are frozen.

Views and prompts. The sheet is a grid of 32-pixel cells, each blank or a digit, rendered from a fixed glyph set. A view is a vertical strip of three cells, which the vision encoder maps to three image tokens. A step prompt consists of an operation token, the view, `=` and two placeholder tokens; the step's outputs are decoded at these three positions in a single forward pass. Since the model is causal, padding after `=` does not affect the outputs (Lemma 4), and steps of different operations can be batched (Methods).

Tables. For each operation, a table enumerates every view the operation can encounter, with its correct outputs: column addition (`SUM`, 242 views); the five operations of long multiplication (1,553); subtraction (`SUB`, 242); estimation of a quotient digit (`EST`, 400); and decrement of an estimate (`DEC`, 9). With $\mathcal{O}$ the nine operations and G the eleven glyphs, the table rows form the set

$$V = \bigsqcup_{o \in \mathcal{O}} V_o, \qquad V_o \subseteq \{o\} \times G^3, \qquad |V| = 242 + 1{,}553 + 651 = 2{,}446, \tag{1}$$

whose terms count addition, multiplication and the calculator tables `SUB`, `EST` and `DEC` (Extended Data Table 1).

Runtime. For each procedure, a runtime of a few hundred lines lays out the operands, extracts views, invokes the model and writes its outputs. In multiplication, control flow is determined entirely by the model's next-operation output. In division and square roots, the runtime fixes the outer loop, and the branches within it are decided by the model's outputs. The runtime checks that every extracted view is a row of a table; no view outside the tables occurred in any experiment. Independent procedures are executed in lockstep, with one view from each in a shared batch. On the M4, a computation of n_{pass} forward passes over n_{view} views takes

$$t \approx \alpha\, n_{\text{pass}} + \beta\, n_{\text{view}}, \qquad \alpha = 0.065 \text{ s}, \quad \beta = 0.037 \text{ s} \tag{2}$$

(Fig. 2c); batching reduces n_{pass} and leaves n_{view} unchanged.

Training. The adapter is trained directly on the tables with cross-entropy, a hinge loss on the logit margin and a distillation penalty toward the base model's distribution on plain text[21] (objective (10) in Methods). Rows whose margin falls below 2 are replayed, and training stops when every row has a margin of at least 3.

3 Certification

At output position k of a view v, let $z_k(v) \in \mathbb{R}^{\Sigma}$ be the model's logits over the vocabulary Σ and $T_k(v)$ the correct token. The margin of v is the smallest lead of a correct token over any other token,

$$m(v) = \min_k \Big(z_k(v)_{T_k(v)} - \max_{a \neq T_k(v)} z_k(v)_a \Big), \tag{3}$$

and v passes if $m(v) > 0$; the model's output $f(v)$, the argmax at each position, then equals $T(v)$ under any tie rule. A certificate records $m(v)$ for every $v \in V$.

Let C be the configurations of a procedure (sheet contents, view position and control state), $\mathcal{V} \supseteq V$ all views and A the outputs. At configuration c_t, the runtime either halts, $\text{view}(c_t) = \perp$, or presents the view $\text{view}(c_t) \in \mathcal{V}$ and writes the outputs:

$$c_{t+1} = \text{paint}\big(c_t,\, f(\text{view}(c_t))\big), \tag{4}$$

where $f\colon \mathcal{V} \to A$ is the model's output map; the reference run uses the table $T\colon \mathcal{V} \to A$ instead. That f is a function of the view alone holds because each step's prompt contains only the operation token and the view (the pinned-sheet policy).

Theorem 1 (Exactness on views). *Let $R_T(I)$ be the set of views presented on reference runs from a set I of initial configurations, and assume that* $\text{paint}(c, \cdot)$ *is injective whenever* $\text{view}(c) \neq \perp$. *The model's*

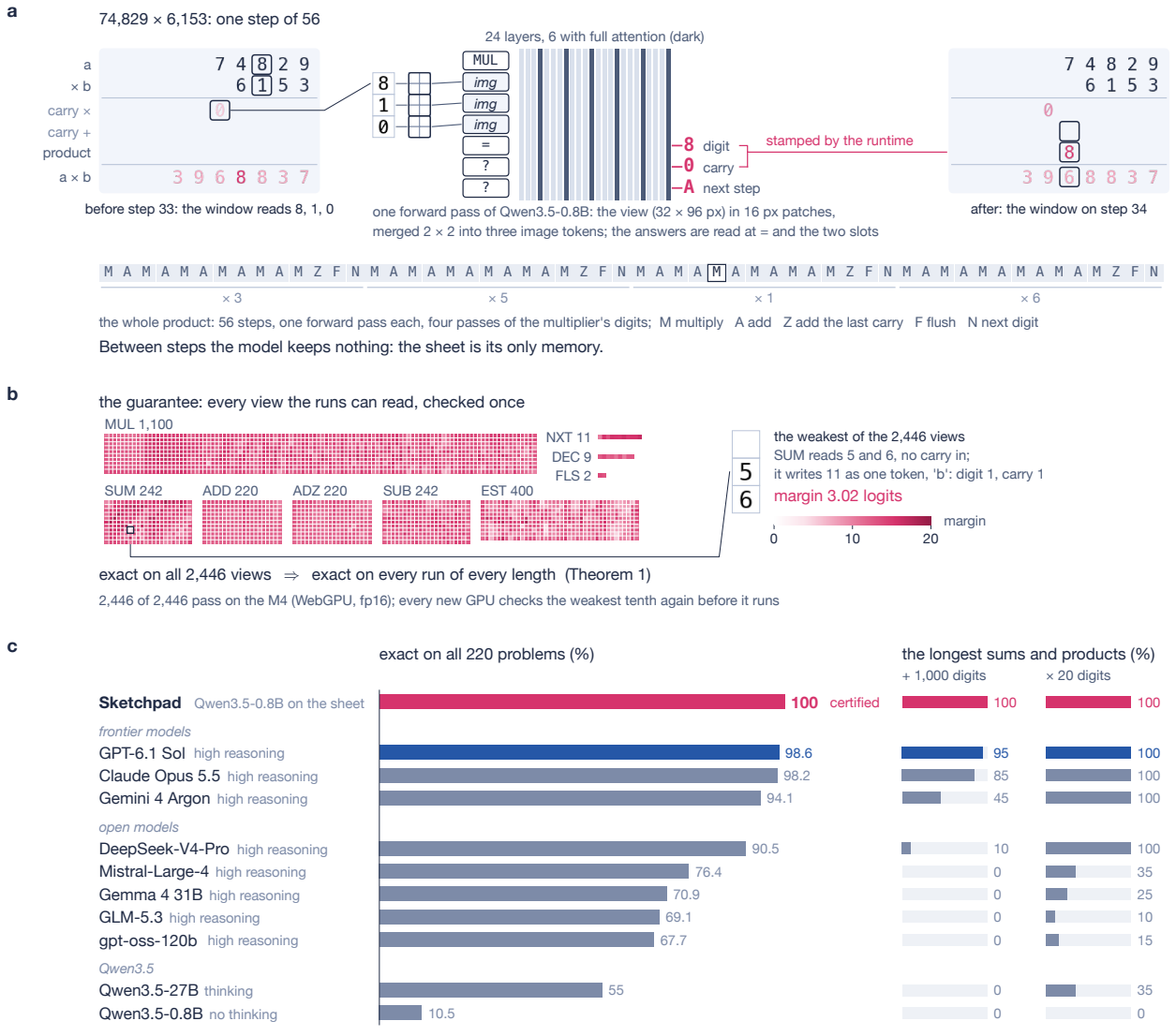


Fig. 1 | Overview. **a**, One step of the long multiplication 74,829 × 6,153 (step 33 of 56). The view (three cells, 32 × 96 pixels) enters Qwen3.5-0.8B as three image tokens after the operation token `MUL`; the outputs, read at `=` and two placeholder positions, are the digit, the carry and the next operation. Below: the operation sequence of the full product. **b**, Margins of all 2,446 views on the deployment device (Apple M4, WebGPU, 16-bit), grouped by table. The smallest margin, 3.02 logits, belongs to the addition view 5 + 6 with no incoming carry. **c**, Exact answers on the pre-registered comparison of 220 problems (sums of 10–1,000 digits and products of 2–20-digit numbers, 20 per length; Methods), overall and on the longest sums and products. Sketchpad in rose, the strongest alternative in cobalt. Per-length results: Extended Data Fig. 1.



Fig. 2 | The sheet machine. **a**, One view and its outputs for each calculator table. **b**, Left: the division 4,100 ÷ 59; the borrow out of the leading column indicates that the estimate 8 is too high. Right: the operation sequence of the same division (Knuth's Algorithm D) and of the square root of 2,000,000 (56 forward passes). **c**, Batching four independent procedures: 32 forward passes instead of 73. Measured time against the cost model for the 200 problems of the browser test (median ratio 1.01). **d**, Time per problem in the browser test, all 200 exact.

run from every $c_0 \in I$ equals the reference run at every step if and only if $f(v) = T(v)$ for all $v \in R_T(I)$. In particular, if $R_T(I) \subseteq V$ and $m(v) > 0$ for all $v \in V$, every run of every length is exact; this direction does not need injectivity.

Proof sketch. If $f = T$ on $R_T(I)$ and the runs agree at c_t, then $\text{view}(c_t) \in R_T(I) \cup \{\perp\}$ and (4) gives the same c_{t+1}; induction on t completes sufficiency. Conversely, if the runs agree and $v = \text{view}(c)$ for a configuration c of a reference run, then $\text{paint}(c, f(v)) = \text{paint}(c, T(v))$, and injectivity gives $f(v) = T(v)$.

The hypothesis that f depends only on the view is essential. If a step's context includes earlier steps, as in chain of thought, no finite test suffices.

Theorem 2 (Necessity of the pinned policy). *Suppose instead that each step's context retains the earlier steps. There is a two-state table and a family of decoders, each a correct lookup plus one attention head, such that for every finite set of test contexts some member passes every row of the table, and every test context that begins with the prompt and ends in a row, with positive margins, yet errs on a longer run. Under the pinned policy the same member is exact at every length.*

Proof sketch. On a context with n tokens after a sink token, k of them events, the head writes $4Rk/(e^c + n)$ to the logit of the wrong state, where R is the lookup's margin and c the head's score on the sink. With $e^c = 3M + 5$, for M the length of the longest test context, the write exceeds R only on longer contexts (Appendix A). This is an instance of Moore's result that no finite experiment identifies a machine with an unbounded number of states[15].

Theorem 3 (Composition). *Consider K calls in which the input $x_i = q_i(v)$ of call i depends only on the results v_j of earlier calls and the answer $g(v)$ only on $v_1, \ldots, v_K$; let q_i^*, T_i^* and g^* be the intended inputs, functions and combining program, with results $v_i^* = T_i^*(q_i^*(v^*))$. If every call returns its table's output on the input it received, $v_i = T_i(x_i)$, as Theorem 1 guarantees for a certified procedure, then a wrong answer requires a wrong table, $T_i(q_i^*(v^*)) \neq T_i^*(q_i^*(v^*))$, a wrong transcription, $q_i(v^*) \neq q_i^*(v^*)$, or a wrong combining program, $g(v^*) \neq g^*(v^*)$, and for any distribution of transcripts*

$$\Pr\left[g(v) \neq g^*(v^*)\right] \leq \Pr[\text{some table wrong}] + \Pr[\text{some transcription wrong}] + \Pr\left[g(v^*) \neq g^*(v^*)\right]. \quad (5)$$

Theorem 4 (Robustness to perturbation). *Let $z, \tilde{z} \in \mathbb{R}^O$ over a finite set O, and $y \in O$. If $\|z - \tilde{z}\|_\infty \leq \tau$ and $z_y - \max_{a \neq y} z_a > 2\tau$, then y is the unique maximiser of $\tilde{z}$. Hence, if $R_T(I) \subseteq V$, $m(v) > 2\tau$ for all $v \in V$, and at every step the logits on the execution path for each view $v \in V$ are within τ of $z_k(v)$ at every output position, every run from I is exact.*

Proof sketch. If $a \neq y$ maximised $\tilde{z}$, then $\tilde{z}_a \leq z_a + \tau < z_y - \tau \leq \tilde{z}_y$. In a run, induction on t as for Theorem 1: while the run agrees with the reference run, its view lies in V and each output is the certified one. A cached-prefix implementation of the pinned step computes the logits of the full prompt exactly (Lemma 4), so it is a path with $\tau = 0$ in exact arithmetic.

Appendix A gives complete proofs and the corresponding Lean 4 theorems. Theorem 4 motivates certifying on the deployment path. A 4-bit quantised decoder moved some margins of an earlier adapter from $+9.4$ to -8.9, whereas the 16-bit decoder reproduced the server certificate; we therefore deploy the 16-bit decoder.

We certify twice. In 32-bit PyTorch, the smallest margin over all 2,446 views is 3.96. On the deployment path (ONNX Runtime Web on WebGPU[22–24], 16-bit, in the batched form used at inference), all views pass with smallest margins of 3.02 for addition, 6.52 for multiplication and 4.61 for the calculator tables (Fig. 3c); the full check takes 3.5 minutes on the M4. The certificate is bound to the model files by their hashes. Before running, the client recomputes the lowest-margin tenth of each table on the local GPU; the full certificate can be recomputed on request.

4 Arithmetic

Signed addition and subtraction proceed column by column; the borrow out of the final column determines the sign. Multiplication is the schoolbook algorithm. Division follows Knuth's Algorithm D[25]. Both operands are multiplied by $\lfloor 10/(d_0 + 1) \rfloor$, where d_0 is the divisor's leading digit; this leaves the quotient unchanged and makes the divisor's leading digit d at least 5. Let $r < 10b$ be the current partial remainder for the scaled

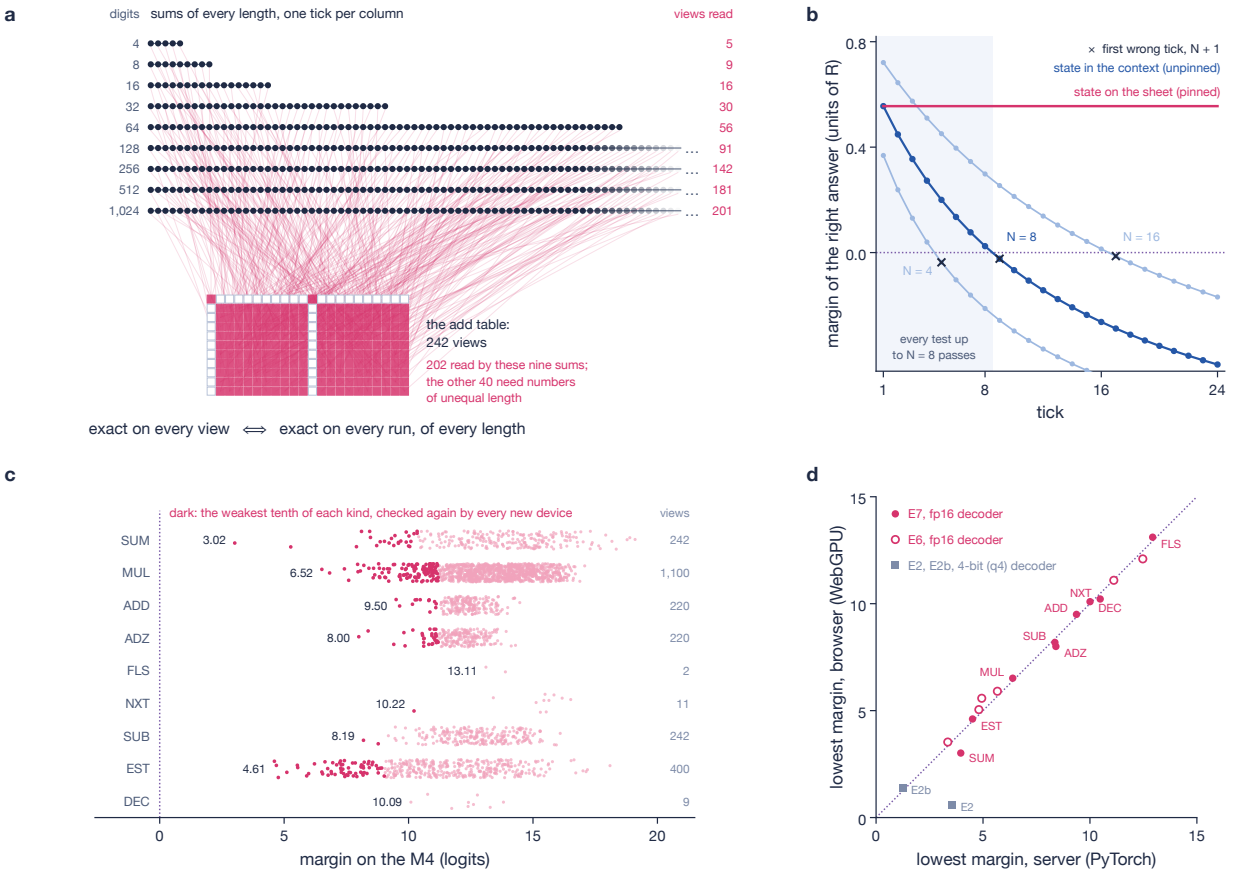


Fig. 3 | Certification. **a**, Sums of increasing length read only views from the 242-view addition table; nine sums of 4 to 1,024 digits reach 202 of them. **b**, The counterexample of Theorem 2: with state in the context, a decoder passes all tests up to N steps and fails at step $N + 1$ ($N = 4$, 8, 16); under the pinned policy the margin is constant. **c**, Margins of all 2,446 views on the M4, by operation; dark points mark the lowest tenth, which is rechecked on each new device. **d**, Smallest margin per operation on the server (PyTorch, 32-bit) against the browser (WebGPU): 16-bit decoders lie on the identity line; 4-bit decoders of earlier adapters deviate by up to 3 logits.

divisor b, with leading digits $r_1 r_2$ when written with one digit more than b, so that $r_1 \le d$. The quotient digit is $q = \lfloor r/b \rfloor$, and `EST` returns

$$\hat{q} = \min\left(9, \left\lfloor \frac{10r_1 + r_2}{d} \right\rfloor\right), \qquad q \le \hat{q} \le q + 2, \tag{6}$$

where the bounds are Knuth's Theorems A and B[25] for base 10 and $d \ge 5$. A borrow out of the multiply-subtract pass $r - \hat{q}\,b$ signals $\hat{q} > q$ and triggers a decrement (`DEC`) and an add-back, at most twice. Square roots use the digit-by-digit method on $c^2 a$, with $c \in \{1, 2, 4, 5\}$ chosen so that the root of $c^2 a$ begins with 25 to 49. Once the partial root has two digits, twice it begins with a digit of at least 5, so the same estimation table applies to every later digit, and $\lfloor \sqrt{a} \rfloor = \lfloor \lfloor \sqrt{c^2 a} \rfloor / c \rfloor$. Greatest common divisors use Euclid's algorithm.

Browser test. We pre-registered 200 problems: 50 each of division to 10 decimal places, square roots to 6 places, greatest common divisors of integers up to 999, and signed sums and differences of up to 20 digits. Run sequentially in the browser on the M4, all 200 answers were exact and every view was a table row. Median times were 7.0 s (division), 9.8 s (square root), 7.0 s (gcd) and 2.0 s (sums); 184 problems finished within 12 s, the slowest in 16.5 s (Extended Data Table 2).

Comparison. We compared Sketchpad with frontier and open models on a pre-registered set of 220 problems: sums of 10 to 1,000 digits and products of 2- to 20-digit numbers, 20 per length, with the same prompt and a budget of 32,768 output tokens including reasoning (Methods). Sketchpad answered all 220 (64,700 views in 1,001 batched forward passes). GPT-6.1 Sol answered 217, Claude Opus 5.5 216 and Gemini 4 Argon 207, all at high reasoning effort; all three were exact on every product, and every error occurred on sums of 300 or 1,000 digits. The errors of GPT-6.1 Sol and Claude Opus 5.5 occurred well within the token budget and differed from the correct sum in 1 to 637 digits; 9 of the 13 errors of Gemini 4 Argon exhausted the budget. Five open-weight models with reasoning enabled answered between 149 (gpt-oss-120b) and 199 (DeepSeek-V4-Pro); most of their errors exhausted the budget on long sums and on products of 15 and 20 digits, except those of Gemma 4 31B, 46 of whose 64 errors were incorrect answers in completed replies. Qwen3.5-27B with chain of thought answered 121, with 93 of its 99 failures exhausting the budget, and Qwen3.5-0.8B answered 23 without chain of thought and 3 with it (Fig. 1c; Extended Data Fig. 1 and Table 5).

5 Algebra

A symbolic runtime solves, expands and factors polynomials by the standard textbook procedures, obtaining every number from the certified procedures. It expands products and collects terms, solves linear equations by rearrangement and quadratics by the quadratic formula, expands powers by the binomial theorem with coefficients from Pascal's rule, factors by the rational root theorem and synthetic division, and solves linear systems by elimination (Fig. 4a). Irrational roots are returned as reduced radicals, for example $3 \pm \sqrt{5}$.

Every answer is checked before it is returned. Rational roots are substituted into the equation, expansions are evaluated at an additional point, and irrational root pairs are verified through Vieta's relations in integer arithmetic. The checks protect the parts the certificate does not cover: the symbolic program and the transcription of the problem. Under fault injection, they rejected 200 of 200 incorrect roots and 185 of 200 runs with one corrupted arithmetic result; in the remaining 15 runs the corruption did not affect the answer (Fig. 4b).

We compare this route with evaluation-based methods from computer algebra, interpolation[26] and Kronecker substitution[27], which also reduce algebra to arithmetic. Under the cost model, the textbook route is faster on ten of the eleven benchmark problems and equal on the eleventh, a plain multiplication: 2.8 s against 4.5 s for a linear equation, 12.1 s against 33.1 s for $(x+2)^5$ and 16.9 s against 31.5 s for factoring $x^3 - 6x^2 + 11x - 6$. Measured on the M4, eight of the eleven finish within 12 s; the remaining three, $(x+1)^{10}$, a cubic factorisation and a 3×3 system, take 16 to 22 s (Fig. 4c; Extended Data Table 3).

6 Gating

An earlier arithmetic adapter changed perplexity on plain text by a factor of only 1.0065, yet reduced strict instruction-following accuracy on IFEval[28] by 16.4 points, an instance of catastrophic forgetting[29,30]. We therefore multiply every adapter branch by a scalar graph input g:

$$y = xW + g\,s\,(xA^\top)B^\top, \qquad y\big|_{g=0} = xW, \tag{7}$$

where W is the frozen weight, A and B are the adapter matrices and s is the LoRA scale (Fig. 5a). At $g = 0$ each branch contributes an exact floating-point zero. On ONNX Runtime, every logit, hidden state

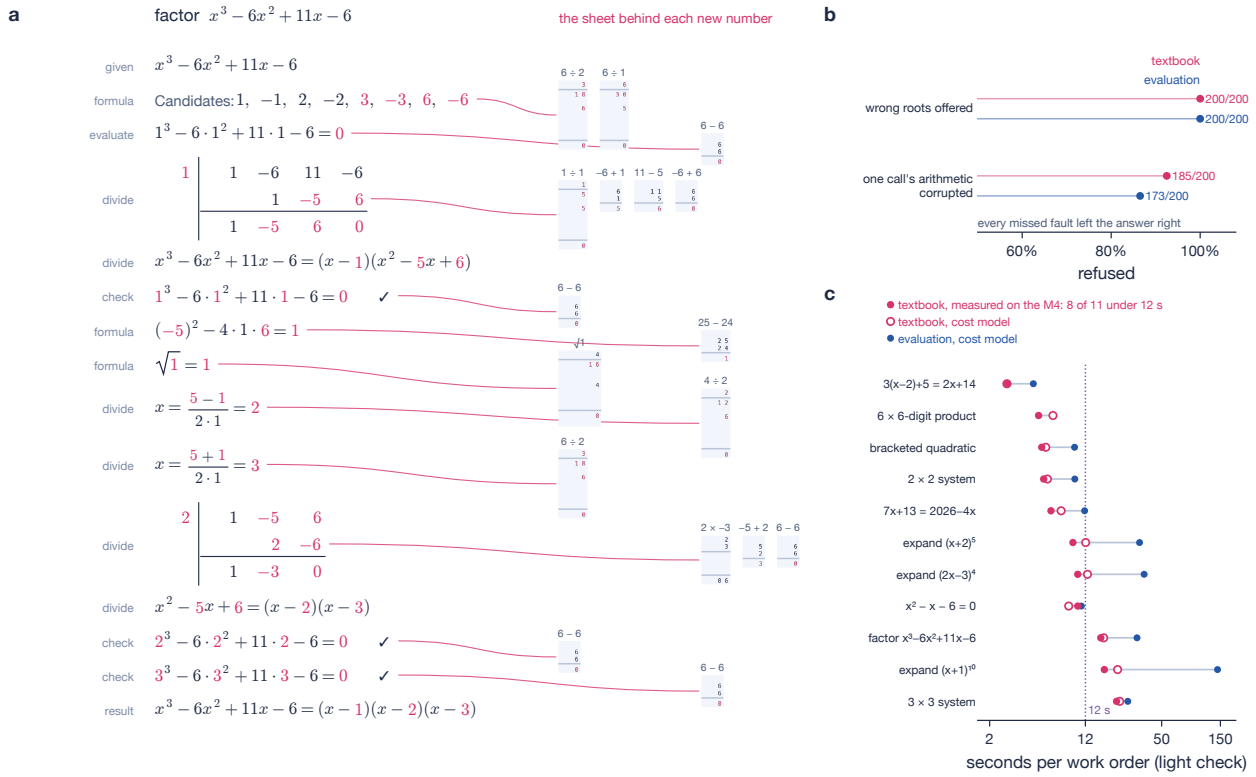


Fig. 4 | Algebra. a, Factorisation of $x^3 - 6x^2 + 11x - 6$. Numbers in rose were computed by the model; each is linked to the column operation that produced it. **b**, Rejection rates under fault injection: incorrect roots (200 of 200 for both routes) and one corrupted arithmetic result (185 of 200 for the textbook route, 173 of 200 for evaluation). **c**, Time per benchmark problem: textbook route measured on the M4 (rose) and under the cost model (open rose); evaluation route under the cost model (cobalt).

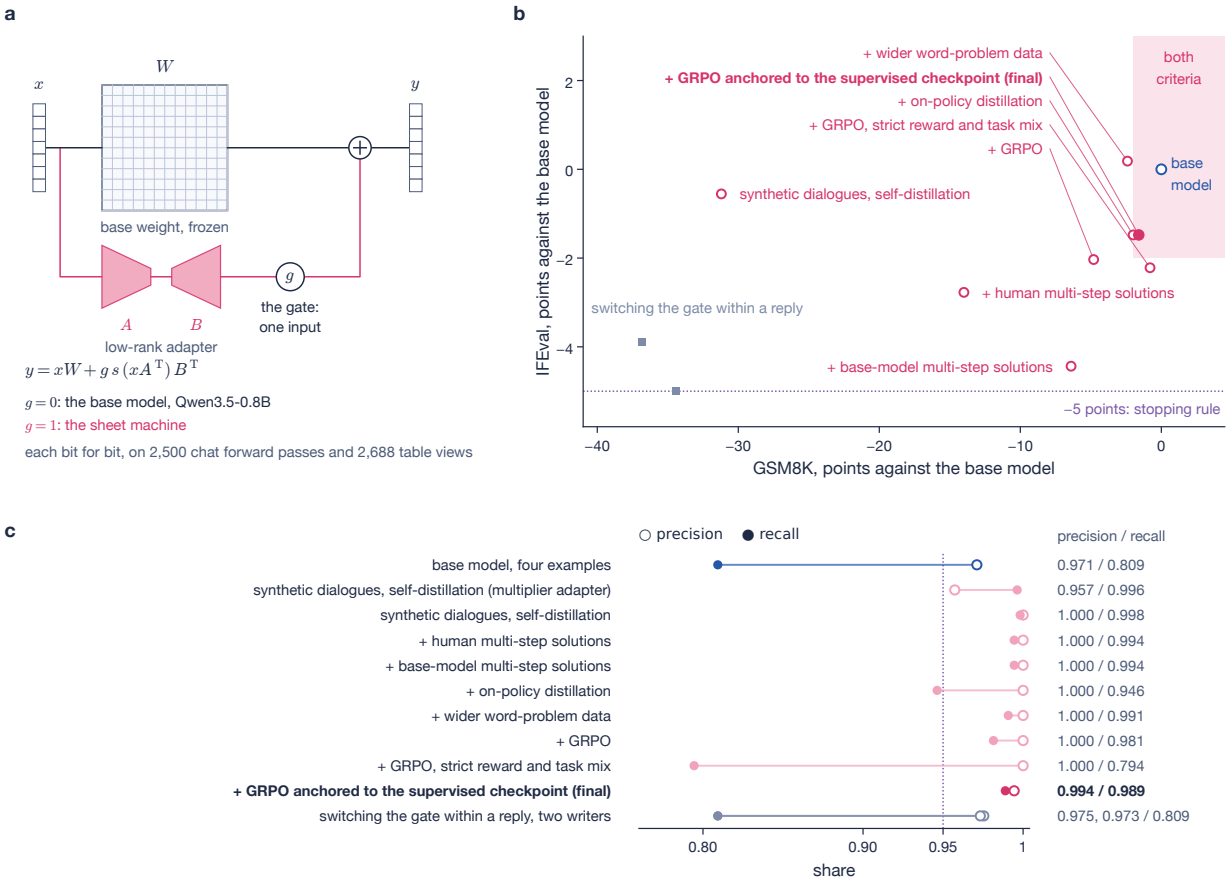


Fig. 5 | Gating. a, A scalar input g multiplies every adapter branch; at $g = 0$ the model is identical to the base model, at $g = 1$ it executes the sheet procedures. **b**, IFEval against GSM8K, in points relative to the base model, for the dialogue fine-tunes and for switching the gate within a reply; the shaded region satisfies both criteria, and the dotted line at -5 points is the pre-registered stopping rule. **c**, Precision and recall of invoking the sheet on held-out problems: base model with four examples (cobalt) and fine-tunes (rose).

and cached state of the gated graph equals that of the base model at $g = 0$ and that of the ungated adapter at $g = 1$, over 2,500 chat forward passes and 2,688 table views; greedy completions of 100 chat prompts are identical. On the M4, the certificate at $g = 1$ reproduces all 2,446 margins.

The gate does not decide when to compute. We therefore trained the adapter to invoke the sheet from within dialogue: on synthetic dialogues and grade-school problems, it learns to emit a short program for the sheet when a reply requires exact computation, and the results are returned into the reply. Training combines supervised data with self-distillation[21] and on-policy distillation[31,32] toward the gated-off model, which limits forgetting[33], followed by reinforcement learning with GRPO[34] against an execution-based reward, with a KL penalty toward the supervised checkpoint. Success criteria were fixed before training. The final adapter meets all of them: on held-out phrasings it invokes the sheet with precision 0.994 and recall 0.989 (the base model prompted with four examples: 0.971 and 0.809), writes the correct computation for 92.8% of word problems, 96.7% of equations and 98.3% of expressions embedded in prose, and stays within 1.6 points of the base model on IFEval (-1.5), GSM8K[35] (-1.6) and MMLU[36] (-1.4), with all 2,446 views still certified; on the M4 the final adapter passes every view with smallest margins of 6.67 (addition), 6.53 (multiplication) and 4.56 (calculator tables), and the same weights at $g = 1$ serve both the arithmetic procedures and dialogue. Intermediate recipes traded instruction following against multi-step reasoning; on-policy distillation removed most of that trade-off, and an unanchored reinforcement-learning stage reintroduced it (Fig. 5b,c; Extended Data Table 4).

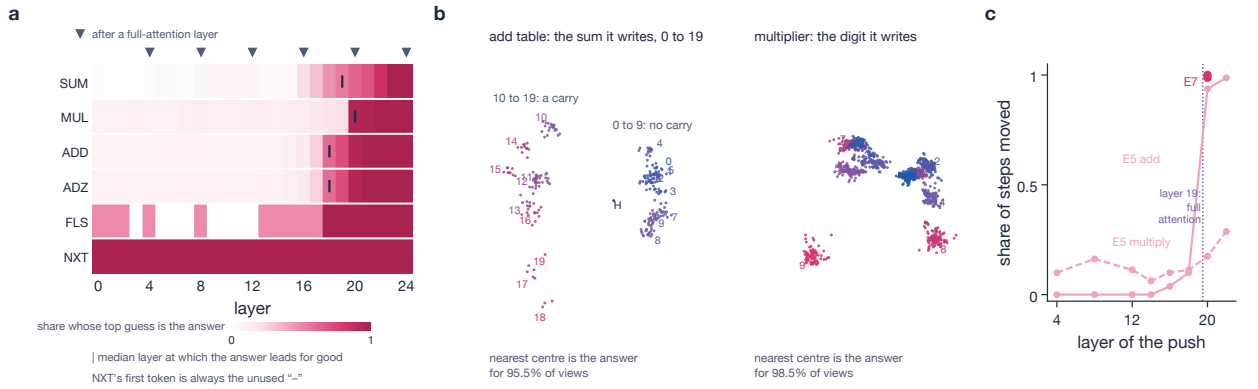


Fig. 6 | Mechanism. **a**, Logit lens: fraction of views whose top token after each depth equals the final output, by operation; ticks mark the median decision depth, triangles the ends of full-attention layers. **b**, Final-layer states projected to three dimensions, coloured by output, for addition (0–19) and multiplication digits. **c**, Fraction of steps changed by a class-mean intervention, against intervention depth, for an earlier adapter (addition and multiplication) and the published adapter (depth 20).

7 Mechanism

We applied the logit lens[37,38] at the first output position: with h_ℓ the residual state after depth ℓ, and W_U and norm the final unembedding and normalisation,

$$\hat{p}_\ell = \mathrm{softmax}\left(W_U\,\mathrm{norm}(h_\ell)\right). \tag{8}$$

The correct output becomes and remains the top token at a median depth of 19 for addition, 20 for multiplication and 18 for the addition steps of the multiplier; up to depth 15 it is the top token for at most 13% of views in any of these tables (Fig. 6a). At the final layer, states cluster by output, and sums with a carry (10–19) separate from those without (Fig. 6b). A class-mean intervention[39,40] moves the state of a view with output a toward a target output a',

$$h_\ell \leftarrow h_\ell + \mu^\ell_{a'} - \mu^\ell_a, \tag{9}$$

where μ^ℓ_a is the mean of h_ℓ over the table's views with output a. Applied after depth $\ell = 20$, the end of the fifth full-attention layer, it changes the output to a' in 79 of 80 addition steps and 80 of 80 multiplication steps. In an earlier adapter, the same intervention after depth 18 changed 8 of 80 addition steps, and after depth 20 it changed 75 of 80, which locates the decision in the full-attention layer ending at depth 20 (Fig. 6c).

8 Related work

Neural arithmetic. The Neural GPU learned binary addition and multiplication that generalise beyond the training length[41,42], and specialised arithmetic units followed[43,44]. For transformers, positional encodings and data formats determine length generalisation[3–9], and fine-tuned models reach high accuracy on bounded arithmetic[45,46], with division and gcd remaining difficult[47–49]. Neural cellular automata and differentiable transducers trained from scratch add and multiply far beyond their training lengths[50–52]. Mechanistic analyses find addition circuits correct on almost all inputs[53], and exactness has been proved under idealised training[54]. Sketchpad differs in using a pretrained vision-language model, in covering division, square roots and gcd, and in certifying exactness by enumeration rather than estimating it.

Procedures in language models. CAEF fine-tunes Llama-3.1-8B to execute arithmetic operators step by step on a text tape, with one adapter per operator[55]. Other work embeds non-learned arithmetic modules[56,57], compiles programs into transformer weights[58–61], trains transformers to predict program execution traces[62], or lets a language model emulate an interpreter[63].

Tool use. Toolformer[10], PAL, Program of Thoughts, ToRA, ReTool and ToolRL[11–14,64] and recursive language models[65] delegate computation to external interpreters.

Verification. Neural network verification typically bounds outputs over continuous input regions[66]; here the input set is finite. Our theorems specialise results on the identification of finite machines[15,67], the verification

of learned recursive programs[16] and certified universal computation by language models[17,68]. The context discipline resembles the inductive scratchpad[69] and PENCIL[70]. Certification from examples can be hard in general and length-generalisation bounds can be uncomputable[71,72]; the pinned policy avoids these obstacles by making the input set finite.

Visual input and scratchpads. Pixel-based language models[73–75] and visual context compression[76,77] read rendered text, though vision-language models still fail at multiplication from images[78]. Visual scratchpads support multimodal reasoning[79–83]. The name refers to Sutherland's Sketchpad[84].

Algebra. Transformers have been trained for symbolic integration[85], polynomial simplification[86] and Gröbner bases[87], and large models solve school algebra with chain of thought[88,89]. None of these is exact by construction.

Continual learning. Learning without Forgetting[21], progressive networks[90], elastic weight consolidation[30] and adapter routing[91] address forgetting; on-policy distillation and the analysis of Shenfeld et al. relate forgetting to divergence from the base model on its own outputs[31–33]. Our gate is a hard switch in the spirit of progressive networks.

9 Limitations

The certificate covers the model's steps only. The runtime, the choice of procedure and the transcription of a problem are conventional code and are not certified; in algebra, answer checks partly compensate. The theorems assume exact arithmetic; on hardware they hold through the margins, for the path on which the margins were measured, and a client by default rechecks only the lowest-margin tenth of each table. The algebra is limited to polynomials of low degree in one variable and linear systems of up to three equations, and its checks verify the roots found, not that all roots were found. Inputs are rendered in the model's glyph set. Execution is slow, about 0.1 s per step on a laptop GPU. Translating a word problem into a computation is not certified, and the 0.8-billion-parameter model mistranslates about 7% of held-out word problems.

Methods

Model and adapter

The language model of Qwen3.5-0.8B[19] has 24 layers with hidden size 1,024; layers 4, 8, …, 24 use full attention (8 query and 2 key-value heads), the others Gated DeltaNet[20]. The LoRA adapter (rank 16, scale 2) is applied to the query, key, value and output projections of the attention layers, the input and output projections of the Gated DeltaNet layers, and the three projections of each feed-forward block: 150 matrices and 10,223,616 parameters, against 873,438,784 in the base model. For deployment the model is exported to ONNX, with 4-bit token embeddings and a 16-bit vision encoder and language model; the adapter is folded into the decoder graph. Two additional graph inputs implement the gate and the steering intervention, and one additional output returns hidden states.

Views, prompts and tables

Glyphs are fixed 32×32 sprites of blank and the ten digits, so the set of possible views is finite and each view is an exact image. The prompt of a step is its operation token, the 32×96 image, `=` and two placeholder tokens, padded with further placeholders to a common length. The tables are exhaustive over each operation's views; for example, `MUL` covers every multiplicand digit or blank, every multiplier digit and every carry from blank to 8 ($11 \times 10 \times 10 = 1{,}100$ views), and `EST` covers every pair of leading remainder digits for each normalised divisor digit from 5 to 9 (400 views). That the procedures reach no other view follows from the layout and, for division and square roots, from normalisation; the runtime checks it at every step, and a reference implementation checked it on 1,000 random problems per procedure.

Training

Each training step draws a batch R of rows from every table, together with rows replayed from the last certificate. With $z_k(v;\theta)$ the logits at output position k of row v, $K_v \in \{1,3\}$ the number of output positions

and $m_k(v;\theta)$ the margin at position k, the term inside the minimum in (3), the loss is

$$\begin{aligned}\mathcal{L}(\theta) = \frac{1}{|R|}\sum_{v\in R}\frac{1}{K_v}\sum_{k=1}^{K_v}\Big[-\log\operatorname{softmax}\big(z_k(v;\theta)\big)_{T_k(v)} + \max\big(0,\, h - m_k(v;\theta)\big)\Big] \\ + \frac{\lambda}{n}\sum_{j=1}^{n}\mathrm{KL}\big(p_0(\cdot \mid x_{\le j}) \,\big\|\, p_\theta(\cdot \mid x_{\le j})\big),\end{aligned} \tag{10}$$

with hinge target $h = 4$ and $\lambda = 1$, where x is a passage of $n = 128$ tokens of plain text and p_0 the base model[21]. Every 100 steps the full certificate is computed and rows with margins below $h/2 = 2$ are replayed until the next; training stops when every row has a margin of at least 3. The published adapter was initialised from an adapter certified on the addition and multiplication tables, with the subtraction, estimation and decrement tables added. It passed every row at step 1,200 and was stopped at step 1,600 (19.8 minutes on one H100 GPU; learning rate on a cosine schedule from 2×10^{-4} to 2×10^{-5} over 4,000 steps). Its plain-text perplexity is 1.0048 times that of the base model.

Certification

Margins are computed over the full vocabulary of 248,320 tokens. The server certificate uses PyTorch in 32-bit floating point. The device certificate runs the deployed files (transformers.js 4.2.0[23], ONNX Runtime Web on WebGPU) through the inference code path and batched prompt form, and records every margin with the device, library version, file hashes and glyph set (Appendix B). Padding the addition prompt to the batched form changed no margin by more than 0.0018. Two device certificates, with and without the gate, agree on all margins to 10^{-3}.

Procedures and timing

Subtraction ends on a row that reports a final borrow, in which case the operands are swapped and the sign recorded. In division, the multiply-subtract pass interleaves `MUL` and `SUB`. In square roots, the scale c^2 makes the root begin with 25 to 49, which we verified for every radicand from 1 to 999,999. The runtime's choice of scale factors and initial estimates affects the number of steps but not the result, since every estimate is at least the true digit and is lowered only by certified steps. The cost model was fitted on batches of 1 to 32 views on the M4.

The browser test drew its 200 problems from a fixed generator before any run: $a \div b$ to 10 places with $a \le 999$ and $2 \le b \le 99$; $\sqrt{a}$ to 6 places with $2 \le a \le 999$; $\gcd(a, b)$ with $10 \le a, b \le 999$; and signed sums and differences of 1 to 20 digits. It ran in Chrome on an Apple M4, one problem at a time. Answers were checked against arbitrary-precision arithmetic. The test was repeated with the gated adapter, with identical views and outputs.

Algebra

The algebra runtime parses a problem, constructs the derivation and obtains every number through the calculator interface; independent calls are batched. Each line of the derivation records the model calls that produced its numbers. The default check substitutes rational roots into the collected equation and verifies expansions at one additional point; the strict check substitutes into the original expressions. An irrational pair of roots of $ax^2 + bx + c$, written $(p \pm q\sqrt{m})/d$ with integers p, $q > 0$, $m > 0$ and $d > 0$, is accepted only if

$$2ap = -bd \quad \text{and} \quad a\,(p^2 - q^2 m) = c\,d^2, \tag{11}$$

which are Vieta's relations $x_1 + x_2 = -b/a$ and $x_1 x_2 = c/a$ multiplied by ad and ad^2; for $a \neq 0$ they hold if and only if the pair are the two roots. Fault injection planted an incorrect root (200 problems per route) or corrupted one arithmetic result (200 per route), with fixed seeds; a further 200 perturbed radical pairs were all rejected. The reference test suite (264 tests, 2,200 generated problems per route and check mode) uses exact tables in place of the model.

Gating

The gate multiplies each adapter branch before it is added to the frozen projection. The identity test compared, bit for bit, every logit, hidden state and cached state of the base, ungated and gated graphs under

ONNX Runtime 1.31.1 on CPU, on 100 chat prompts (first user turns of the No Robots test split[92]; one prefill and 24 greedy steps each) and on all table views.

Dialogue fine-tuning

The training data comprise about 3,150 synthetic dialogues, about 30% of which should not invoke the sheet, and 547 grade-school problems solved by the base model with each calculation replaced by a call to the sheet. The held-out set contains 780 items from unseen templates. Invocations are scored by execution. Criteria were fixed before training: precision and recall of invocation of at least 0.95; correct invocations for at least 90% of word problems and 95% of equations; IFEval, GSM8K (250 problems) and MMLU (500 questions) within 2 points of the base model. Reinforcement learning used GRPO with groups of eight samples per task and four tasks per step, a reward that requires a finished reply quoting the executed result and penalises repeated or excess invocations, and a KL penalty toward the supervised checkpoint on every generated token; it ran for 200 steps on one H100 GPU.

Mechanism

The logit lens (8) is computed after each layer at the first output position, for the 1,795 addition and multiplication views. The decision depth of a view is the first depth after which the final output remains the top token. The class-mean intervention (9) acts at the first output position, with the target drawn at random from the table's other outputs (80 trials per table).

Baselines

The comparison was pre-registered with its scoring rule. Problems were generated with a fixed seed; every model received the prompt "What is a + b? Give the exact integer. Put the final answer in \boxed{}." (with $\times$ for products). The answer was the last boxed integer, and a reply that exhausted the budget or gave no answer was scored as wrong. Qwen3.5-0.8B and Qwen3.5-27B (FP8 release) ran under vLLM with the recommended sampling settings and budgets of 32,768 tokens with chain of thought and 4,096 without. GPT-6.1 Sol (`gpt-6.1-sol`, reasoning effort high), Claude Opus 5.5 (`claude-opus-5-5`, effort high) and Gemini 4 Argon (`gemini-4-argon`, thinking level high) were accessed through their APIs on 31 October 2026 with default sampling and a budget of 32,768 output tokens including reasoning. Five open-weight models, DeepSeek-V4-Pro (`deepseek-v4-pro:0813`), Mistral-Large-4, Gemma 4 31B, GLM-5.3 and gpt-oss-120b, ran through the Ollama cloud API on 31 October 2026 with reasoning enabled at the highest available setting and the same budget; Kimi K3 was piloted and excluded on cost. Sketchpad ran all 220 problems in batched lockstep in PyTorch.

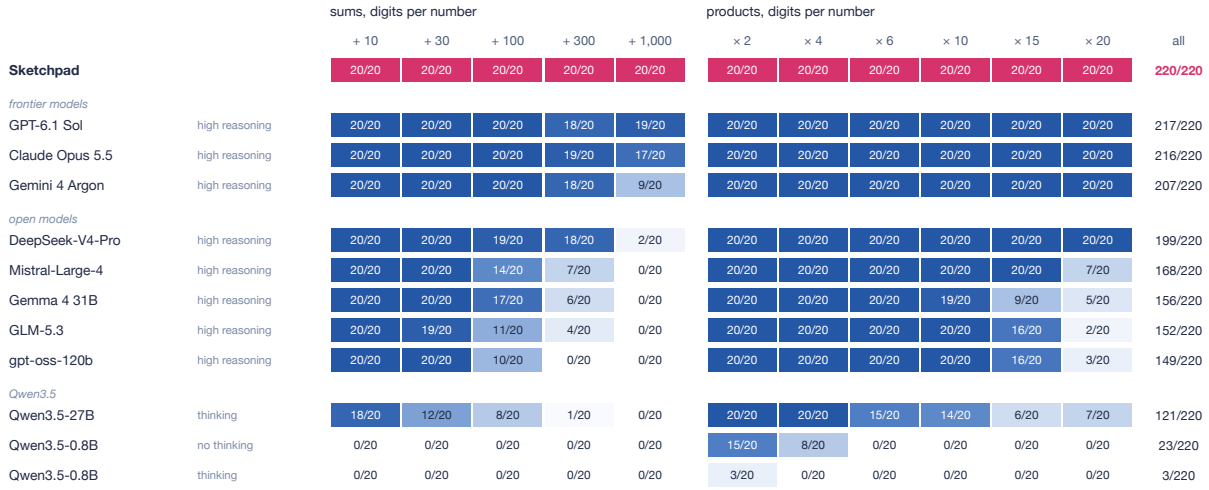
Data and code availability

The model, adapter, certificate and analysis data are available on Hugging Face (`mohsenvand/sketchpad-0.8b-preview`). The runtime, the algebra, the baselines and the sources of this paper are available under the Apache 2.0 licence in the Sketchpad repository. The Lean proofs are in the accompanying package (Appendix A).

Extended Data

A Proofs

The theorems are proved in the Lean 4 proof assistant[93] with Mathlib[94] (version 4.35.0), without `sorry` or additional axioms; `#print axioms` reports only `propext`, `Classical.choice` and `Quot.sound`, or a subset. The formalisation models the network as a deterministic function of its context, or as a causal decoder whose layer at a position reads only earlier positions, with arbitrary tie-breaking in the argmax and exact arithmetic; floating point enters only through τ in Theorem 4. The proofs below follow the Lean proofs, whose declarations are listed in Extended Data Table 6 (in the namespace `Polyformer.Pinned` unless prefixed). Theorem 1 is formalised for the sheet machine (`Sheet.lean`) and for a token form, in which the state is a token and each step reads a state token and an event token after a fixed prompt (`Pinned.lean`). Lemmas



Extended Data Fig. 1 | Exact answers by length. Problems answered exactly, of 20 per length, for every system in the comparison: sums of 10 to 1,000 digits (left) and products of 2- to 20-digit numbers (right).

Extended Data Table 1 | Tables. Each row is one view an operation can encounter. Margins are the minimum over the table, in logits, on the server (PyTorch, 32-bit) and on the M4 (WebGPU, 16-bit). Next-operation codes: `A` add the partial product, `Z` end of pass, `M` multiply, `F` flush, `N` next digit, `H` halt, `B` final borrow, `S` continue subtraction, `C` correct the remainder.

Operation	View (top to bottom)	Outputs at `=`, `?`, `?`	Rows	Server	M4
`SUM`	carry (blank, 1), top digit, bottom digit (blank, 0–9)	sum and carry as one token (0–19), or `H`	242	3.96	3.02
`MUL`	multiplicand digit (blank, 0–9), multiplier digit (0–9), carry (blank, 0–8)	digit, carry, `A` or `Z`	1,100	6.39	6.52
`ADD`	carry (blank, 1), product digit, running-total digit (blank, 0–9)	digit, carry, `M`	220	9.37	9.50
`ADZ`	as `ADD`, at the end of a pass	digit, carry, `F`	220	8.41	8.00
`FLS`	carry (blank, 1), blank, blank	final carry, –, `N`	2	12.93	13.11
`NXT`	next multiplier digit (blank, 0–9), blank, blank	–, –, `M` or `H`	11	10.48	10.22
`SUB`	borrow (blank, 1), top digit, bottom digit (blank, 0–9)	digit, borrow, `S`; all blank: –, –, `H` or `B`	242	8.36	8.19
`EST`	leading remainder digits $r_1 \le d$, leading divisor digit d (5–9)	$\min(9, \lfloor(10r_1 + r_2)/d\rfloor)$, –, `M` or `N`	400	4.51	4.61
`DEC`	estimate q (1–9), blank, blank	$q - 1$, –, `C`	9	10.00	10.09
All			**2,446**	**3.96**	**3.02**

Extended Data Table 2 | Browser test. 50 problems per set on the Apple M4 (WebGPU, 16-bit decoder).

Set	Exact	Median s	Max s	Over 12 s	Median passes / views	Cost model median s
$a \div b$ to 10 places ($a \le 999$, $b \le 99$)	50	7.0	13.3	1	65 / 90	7.6
$\sqrt{a}$ to 6 places ($a \le 999$)	50	9.8	16.5	11	86 / 137	10.7
$\gcd(a, b)$ ($a, b \le 999$)	50	7.0	13.4	4	61 / 84	7.1
signed $\pm$, up to 20 digits	50	2.0	4.7	0	19 / 19	1.9

Extended Data Table 3 | Cost of the algebra benchmark. Views, forward passes after batching and seconds under the cost model, default check, for the textbook and evaluation routes.

Problem	Textbook views / passes / s	Evaluation views / passes / s
`solve 3(x-2)+5=2x+14`	36 / 22 / 2.8	75 / 27 / 4.5
`solve 7x+13=2026-4x`	97 / 62 / 7.6	165 / 88 / 11.8
`solve 2x+3y=7, x-y=1`	75 / 48 / 5.9	194 / 41 / 9.8
`solve (2x-3)(x+5)=x^2+1-16`	79 / 43 / 5.7	177 / 50 / 9.8
`solve x^2-x-6=0`	102 / 77 / 8.8	162 / 79 / 11.1
`123456 * 654321`	96 / 46 / 6.5	96 / 46 / 6.5
`expand (x+2)^5`	159 / 95 / 12.1	458 / 248 / 33.1
`expand (2x-3)^4`	172 / 94 / 12.5	492 / 274 / 36.0
`factor x^3-6x^2+11x-6`	195 / 149 / 16.9	483 / 210 / 31.5
`expand (x+1)^10`	322 / 154 / 21.9	2,359 / 845 / 142.2
`solve 2x+3y-z=1, x-y+2z=3, 3x+y+z=8`	323 / 167 / 22.8	538 / 102 / 26.5
`solve x^2-2=0`	86 / 65 / 7.4	
`solve 2x^2+3x-1=0`	144 / 88 / 11.0	
`solve x^2-6x+4=0`	133 / 96 / 11.2	

Extended Data Table 4 | Dialogue fine-tuning. Held-out templates with the gate on and no examples, against the base model prompted with four examples (invocation and translation) or with none (benchmarks). IFEval, GSM8K and MMLU in points relative to the base model (0.536, 0.516 and 0.476).

Recipe	Precision / recall	Word	Equations	Prose	IFEval	GSM8K	MMLU
criterion	≥ 0.95 / ≥ 0.95	≥ 0.90	≥ 0.95	≥ 0.95	≥ -2	≥ -2	≥ -2
base model, four examples	0.971 / 0.809	0.789	0.583	0.667	0	0	0
synthetic dialogues, self-distillation (multiplier adapter)	0.957 / 0.996	0.872	0.967	0.983	−4.8		−6.6
synthetic dialogues, self-distillation	1.000 / 0.998	0.894	0.975	1.000	−0.55	−31.2	−5.4
+ human multi-step solutions	1.000 / 0.994	0.856	0.975	0.992	−2.8	−14.0	−2.4
+ base-model multi-step solutions	1.000 / 0.994	0.817	0.958	0.992	−4.4	−6.4	−3.4
+ on-policy distillation	1.000 / 0.946	0.678	0.958	0.983	−1.5	−2.0	−2.2
+ wider word-problem data	1.000 / 0.991	0.856	0.958	0.975	+0.2	−2.4	−2.2
+ GRPO	1.000 / 0.981	0.861	0.958	0.992	−2.0	−4.8	−0.6
+ GRPO, strict reward and task mix	1.000 / 0.794	0.378	0.933	0.975	−2.2	−0.8	−1.0
+ GRPO anchored to the supervised checkpoint (final)	**0.994 / 0.989**	**0.928**	**0.967**	**0.983**	**−1.5**	**−1.6**	**−1.4**

Extended Data Table 5 | Comparison. Problems answered exactly, of 20 per cell; in brackets, the number of replies that exhausted the token budget. Qwen3.5-27B is the FP8 release; all other models ran with reasoning at their highest available setting, accessed on 31 October 2026.

System	+10	+30	+100	+300	+1,000	×2	×4	×6	×10	×15	×20	All
Sketchpad	20	20	20	20	20	20	20	20	20	20	20	220
GPT-6.1 Sol	20 (0)	20 (0)	20 (0)	18 (0)	19 (0)	20 (0)	20 (0)	20 (0)	20 (0)	20 (0)	20 (0)	217
Claude Opus 5.5	20 (0)	20 (0)	20 (0)	19 (0)	17 (0)	20 (0)	20 (0)	20 (0)	20 (0)	20 (0)	20 (0)	216
Gemini 4 Argon	20 (0)	20 (0)	20 (0)	18 (1)	9 (8)	20 (0)	20 (0)	20 (0)	20 (0)	20 (0)	20 (0)	207
DeepSeek-V4-Pro	20 (0)	20 (0)	19 (0)	18 (0)	2 (17)	20 (0)	20 (0)	20 (0)	20 (0)	20 (0)	20 (0)	199
Mistral-Large-4	20 (0)	20 (0)	14 (5)	7 (10)	0 (20)	20 (0)	20 (0)	20 (0)	20 (0)	20 (0)	7 (13)	168
Gemma 4 31B	20 (0)	20 (0)	17 (0)	6 (2)	0 (15)	20 (0)	20 (0)	20 (0)	19 (1)	9 (0)	5 (0)	156
GLM-5.3	20 (0)	19 (1)	11 (6)	4 (15)	0 (20)	20 (0)	20 (0)	20 (0)	20 (0)	16 (3)	2 (18)	152
gpt-oss-120b	20 (0)	20 (0)	10 (9)	0 (18)	0 (20)	20 (0)	20 (0)	20 (0)	20 (0)	16 (4)	3 (17)	149
Qwen3.5-27B, chain of thought	18 (2)	12 (8)	8 (10)	1 (19)	0 (20)	20 (0)	20 (0)	15 (4)	14 (6)	6 (11)	7 (13)	121
Qwen3.5-0.8B, chain of thought	0 (20)	0 (20)	0 (20)	0 (20)	0 (20)	3 (17)	0 (20)	0 (20)	0 (20)	0 (20)	0 (20)	3
Qwen3.5-0.8B	0 (8)	0 (20)	0 (20)	0 (20)	0 (20)	15 (4)	8 (9)	0 (14)	0 (20)	0 (19)	0 (20)	23

Extended Data Table 6 | Lean statements. Files are in the package's `Polyformer/` folder, except `Sheet.lean`, which is in `paper/lean/` of the Sketchpad repository.

Statement	Lean declarations	File
Theorem 1	`Sketchpad.sheet_exact_iff`, `Sketchpad.sheet_certified`; token form `regeneration_trace_iff`, `pinned_regeneration_iff`	`Sheet.lean`, `Pinned.lean`
Theorem 2	`no_finite_test_suite`, `theorem8a_counterexample`, `no_finite_check`	`Unpinned.lean`
Theorem 3	(a) `error_decomposition`; (b) token form `compose_exact`, `error_decomposition_certified`, `error_bound`	`Compose.lean`
Theorem 4	`sel_eq_of_close`, `histRun_eq_of_closed`; token form `regeneration_of_close`; unpinned decoding `unpinned_exact_of_close`	`Pinned.lean`, `Unpinned.lean`
Lemma 1	`sel_eq_of_close`, `sel_eq_of_strict`	`Pinned.lean`
Lemma 2	`Polyformer.Interp.regeneration`, `scanl_eq_iff_foldl_eq`, `regeneration_trace_iff`	`Interp.lean`, `Pinned.lean`
Lemma 3	`histRun_eq_of_closed`, `reach_closed`	`Pinned.lean`
Lemma 4	`resid_congr`, `cache_sound`, `cachedForward_eq`, `policy_tick_eq`	`Pinned.lean`, `Policy.lean`

1–4 and Theorem 3a are formalised in general. The run-level statements of Theorems 3b and 4 are formalised for the token form; for the sheet machine they follow from the lemmas as shown below.

Definitions and lemmas

Definition 1 (Sheet machine). *A sheet machine consists of sets C of configurations, $\mathcal{V}$ of views and A of outputs, a view map* $\mathrm{view}\colon C \to \mathcal{V} \cup \{\bot\}$ *and a paint map* $\mathrm{paint}\colon C \times A \to C$. *For $g\colon \mathcal{V} \to A$, the step $\sigma_g\colon C \to C$ is*

$$\sigma_g(c) = \begin{cases} \mathrm{paint}\big(c, g(v)\big) & \text{if } \mathrm{view}(c) = v \in \mathcal{V}, \\ c & \text{if } \mathrm{view}(c) = \bot, \end{cases} \tag{12}$$

and the run of length n from c_0 is $(c_0, \sigma_g(c_0), \ldots, \sigma_g^n(c_0))$. The model's run and the reference run are those of σ_f and σ_T. For $I \subseteq C$,

$$\mathrm{Reach}_T(I) = \{\sigma_T^n(c_0) : c_0 \in I,\ n \geq 0\}, \qquad R_T(I) = \{\mathrm{view}(c) : c \in \mathrm{Reach}_T(I)\} \setminus \{\bot\}.$$

Definition 2 (Runs over events). *Let S be a set of states, E a set of events and $F\colon S \times E \to S$. For $s_0 \in S$ and a word w over E, $F^w(s_0)$ is the left fold, $F^\varepsilon(s_0) = s_0$ and $F^{ue}(s_0) = F(F^u(s_0), e)$, and the trace of F on $w = e_1 \cdots e_n$ is $\big(F^{e_1 \cdots e_i}(s_0)\big)_{i=0}^n$. For $B \subseteq E$, $\mathrm{Reach}_F^B(s_0) = \{F^u(s_0) : u \in B^*\}$.*

Lemma 1 (Margins decide the argmax). *Let O be a finite set, $z, \tilde{z} \in \mathbb{R}^O$, $y \in O$ and $\tau \in \mathbb{R}$. If $z_a + 2\tau < z_y$ for every $a \in O \setminus \{y\}$ and $|\tilde{z}_a - z_a| \leq \tau$ for every $a \in O$, then every maximiser of $\tilde{z}$ over O is y. In particular, if $m(v) > 0$, then $f(v) = T(v)$ under every tie rule.*

Proof. Let s maximise $\tilde{z}$ over O and suppose $s \neq y$. Then $\tilde{z}_s \leq z_s + \tau < z_y - \tau \leq \tilde{z}_y$, which contradicts $\tilde{z}_y \leq \tilde{z}_s$. For the second statement take, at each output position k, $O = \Sigma$, $y = T_k(v)$, $\tilde{z} = z = z_k(v)$ and $\tau = 0$; the hypothesis is that the k-th term in (3) is positive.

Lemma 2 (Regeneration). *Let $F, \delta\colon S \times E \to S$, $s_0 \in S$ and $B \subseteq E$. Then $F^w(s_0) = \delta^w(s_0)$ for every $w \in B^*$ if and only if $F(s, e) = \delta(s, e)$ for every $s \in \mathrm{Reach}_\delta^B(s_0)$ and $e \in B$. Both conditions are equivalent to equality of the traces of F and δ on every $w \in B^*$.*

Proof. ($\Leftarrow$) Induction on the length of w. For $w = \varepsilon$ both sides are s_0. For $w = ue$ with $u \in B^*$ and $e \in B$,

$$F^{ue}(s_0) = F\big(F^u(s_0), e\big) = F\big(\delta^u(s_0), e\big) = \delta\big(\delta^u(s_0), e\big) = \delta^{ue}(s_0),$$

by the induction hypothesis and then by the assumption at $s = \delta^u(s_0) \in \mathrm{Reach}_\delta^B(s_0)$.

($\Rightarrow$) Let $s = \delta^u(s_0)$ with $u \in B^*$, and let $e \in B$. Applying the assumption to u and to ue,

$$F(s,e) = F\big(F^u(s_0), e\big) = F^{ue}(s_0) = \delta^{ue}(s_0) = \delta(s,e).$$

Traces: the i-th entry of the trace on w is the fold over the prefix $e_1 \cdots e_i \in B^*$, so equal folds on all words give equal traces; conversely, the last entries of equal traces are the folds over w.

Lemma 3 (Steps that read the history). *Let $G\colon E^* \times S \times E \to S$, and for $w = e_1 \cdots e_n$ let $s'_0 = s_0$ and $s'_i = G(e_1 \cdots e_{i-1}, s'_{i-1}, e_i)$. If $G(u,s,e) = \delta(s,e)$ for all $u \in B^*$, $s \in \mathrm{Reach}_\delta^B(s_0)$ and $e \in B$, then $s'_n = \delta^w(s_0)$ for every $w \in B^*$.*

Proof. By induction on i, $s'_i = \delta^{e_1 \cdots e_i}(s_0)$. This holds for $i = 0$. If it holds for $i-1$, then $s'_{i-1} \in \mathrm{Reach}_\delta^B(s_0)$, and the assumption with $u = e_1 \cdots e_{i-1}$ gives $s'_i = \delta(s'_{i-1}, e_i) = \delta^{e_1 \cdots e_i}(s_0)$. Lean proves the lemma with any set $Q \ni s_0$ closed under δ on B in place of $\mathrm{Reach}_\delta^B(s_0)$, which is the smallest such set.

Definition 3 (Causal decoder). *A causal decoder of depth L maps tokens $x_0 x_1 \cdots$ to states $h_p^0 = \mathrm{emb}(x_p, p)$ and $h_p^{\ell+1} = \Lambda_\ell(p, h^\ell)$, where $\Lambda_\ell(p,h)$ depends only on $h_0, \ldots, h_p$; the logits of a sequence of length N are $\mathrm{head}(h_{N-1}^L)$. A cached forward of a block b after a prefix Π reads h_p^ℓ for $p < |\Pi|$ from a cache computed on Π alone and computes the positions $p \geq |\Pi|$ by the same recursion.*

Lemma 4 (Causality and caching). *(i) h_p^ℓ depends only on $x_0, \ldots, x_p$. (ii) The cached forward of b after Π computes $h_p^\ell(\Pi b)$ at every layer and position, and hence the logits of Πb. (iii) Decoding a two-token block $b_1 b_2$ one token at a time on the cache of Π gives the logits of $\Pi b_1 b_2$.*

Proof. (i) Induction on ℓ: h_p^0 reads only x_p; if x and x' agree up to p, the induction hypothesis gives $h_i^\ell(x) = h_i^\ell(x')$ for $i \leq p$, so $h_p^{\ell+1}(x) = h_p^{\ell+1}(x')$ by causality of Λ_ℓ. (ii) Induction on ℓ. For $p < |\Pi|$ the cache holds $h_p^\ell(\Pi)$, which equals $h_p^\ell(\Pi b)$ by (i). For $p \geq |\Pi|$, layer 0 embeds $b_{p-|\Pi|} = (\Pi b)_p$, and layer $\ell+1$ applies $\Lambda_\ell(p,\cdot)$ to states equal to $h^\ell(\Pi b)$ by the induction hypothesis. (iii) Decoding b_1 on the cache of Π leaves the cache of Πb_1 by (ii), and decoding b_2 on that cache gives the logits of $\Pi b_1 b_2$, again by (ii).

Theorem 1

Proof of Theorem 1. Fix $c_0 \in I$ and apply Lemma 2 with $S = C$, a single event $E = B = \{*\}$, $F(c,*) = \sigma_f(c)$ and $\delta(c,*) = \sigma_T(c)$. The words over B are $*^n$, $F^{*^n}(c_0) = \sigma_f^n(c_0)$, the trace on $*^n$ is the run of length n, and $\mathrm{Reach}_\delta^B(c_0) = \mathrm{Reach}_T(\{c_0\})$. By the lemma, the model's run from c_0 equals the reference run at every length if and only if

$$\sigma_f(c) = \sigma_T(c) \quad \text{for every } c \in \mathrm{Reach}_T(\{c_0\}). \tag{13}$$

Sufficiency. Let $f = T$ on $R_T(I)$ and $c \in \mathrm{Reach}_T(\{c_0\})$. If $\mathrm{view}(c) = \bot$, then $\sigma_f(c) = c = \sigma_T(c)$. Otherwise $v = \mathrm{view}(c) \in R_T(I)$ and $\sigma_f(c) = \mathrm{paint}(c, f(v)) = \mathrm{paint}(c, T(v)) = \sigma_T(c)$. So (13) holds for every $c_0 \in I$.

Necessity. Let $v \in R_T(I)$, so that $v = \mathrm{view}(c)$ for some $c \in \mathrm{Reach}_T(\{c_0\})$ and $c_0 \in I$. By (13), $\mathrm{paint}(c, f(v)) = \mathrm{paint}(c, T(v))$, and injectivity of $\mathrm{paint}(c,\cdot)$ gives $f(v) = T(v)$.

The certificate. If $R_T(I) \subseteq V$ and $m(v) > 0$ for all $v \in V$, Lemma 1 gives $f = T$ on $V \supseteq R_T(I)$, and sufficiency, which did not use injectivity, applies.

Theorem 2

Let Σ be a set of tokens, Π a fixed prompt and $\rho\colon \Sigma^* \to \Sigma$ a step map that returns a maximiser of the logits at the last position of a context. On events $e_1 e_2 \cdots$ from a state s_0, pinned decoding computes $s_i = \rho(\Pi\, s_{i-1}\, e_i)$; unpinned decoding keeps the whole context, $x_0 = \Pi\, s_0$, $s_i = \rho(x_{i-1}\, e_i)$ and $x_i = x_{i-1}\, e_i\, s_i$. A row (s,e) of a table δ passes if the logits at $\Pi\, s\, e$ have their strict maximum at $\delta(s,e)$.

Definition 4 (The counterexample). *The tokens are $\Sigma = \{\omega, \mathsf{u}, \mathsf{v}, \mathsf{a}\}$, where ω is a sink token, and the prompt is $\Pi = \omega\,\mathsf{u}\,\mathsf{a}\,\mathsf{u}\,\mathsf{v}\,\mathsf{a}\,\mathsf{v}$. The table has states $\{\mathsf{u}, \mathsf{v}\}$, the event a, $\delta(\mathsf{u},\mathsf{a}) = \mathsf{u}$, $\delta(\mathsf{v},\mathsf{a}) = \mathsf{v}$ and start u, so its state is u after every input. For $R > 0$ and $c \in \mathbb{R}$, the logits at the last position of a context $x = x_0 x_1 \cdots x_{N-1}$ are*

$$\ell(x)_y = R\,[\,y = x_{N-2}\,] + [\,y = \mathsf{v}\,]\,\eta(x), \qquad \eta(x) = \frac{\sum_{j=0}^{N-1} e^{\kappa_j}\,\beta_j}{\sum_{j=0}^{N-1} e^{\kappa_j}}, \tag{14}$$

with scores $\kappa_0 = c$ and $\kappa_j = 0$ for $j \geq 1$, and values $\beta_j = 4R$ if $x_j = \mathsf{a}$ and $\beta_j = 0$ otherwise. The first term is a lookup that returns the state token of the last row, which is the correct output for δ; the second is one attention head at the last position, with score c on the sink, which writes its value to the logit of v. If $x_0 = \omega$ and k of the $n = N - 1$ tokens after it are a,

$$\eta(x) = \frac{4Rk}{e^c + n}. \tag{15}$$

Theorem 2 (Necessity of the pinned policy, precise form). *Let $\mathcal{T}$ be a finite set of contexts, M the largest length of a context in $\mathcal{T}$ ($M = 0$ if $\mathcal{T}$ is empty), and $e^c = 3M + 5$. For every $R > 0$ and every tie rule, the decoder (14): (a) passes both rows of δ with positive margins; (b) at every context in $\mathcal{T}$ of the form $\Pi\, w\, s\, \mathsf{a}$ with $s \in \{\mathsf{u}, \mathsf{v}\}$, has its strict maximum at $\delta(s, \mathsf{a})$; (c) under unpinned decoding of a^{2M+4} ends in the state v, whereas δ ends in u; and (d) under pinned decoding equals δ on every input.*

Proof. Two facts. At a context ending in $\mathsf{v}\,\mathsf{a}$, $\ell_{\mathsf{v}} = R + \eta \geq R > 0$ while every other logit is 0, since $\eta \geq 0$; so v is the strict maximiser. At a context ending in $\mathsf{u}\,\mathsf{a}$, $\ell_{\mathsf{u}} = R$, $\ell_{\mathsf{v}} = \eta$ and $\ell_\omega = \ell_{\mathsf{a}} = 0$; so u is the strict maximiser if $\eta < R$, and v is if $\eta > R$.

(a) A row $\Pi\, s\, \mathsf{a}$ has $n = 8$ tokens after the sink, three of them a, so $\eta = 12R/(e^c + 8)$ by (15), which is less than R because $e^c = 3M + 5 > 4$. Both facts then give the rows.

(b) A test context $x = \Pi\, w\, \mathsf{u}\, \mathsf{a}$ of length $N \leq M$ has $n = N - 1$ tokens after the sink and at most $N - 1$ of them a, so

$$\eta(x) \leq \frac{4R(N-1)}{3M + 4 + N} < R \iff 3N < 3M + 8,$$

which holds because $N \leq M$. Test contexts ending in $\mathsf{v}\,\mathsf{a}$ are covered by the first fact.

(c) A state v is followed only by v, by the first fact, so if $s_{i-1} = \mathsf{u}$, then $s_0 = \cdots = s_{i-1} = \mathsf{u}$ and the context that determines s_i is $\Pi\, \mathsf{u}\, (\mathsf{a}\,\mathsf{u})^{i-1}\, \mathsf{a}$, with $2i + 6$ tokens after the sink, $i + 2$ of them a. By (15),

$$\eta = \frac{4R(i+2)}{e^c + 2i + 6} > R \iff 2i > e^c - 2,$$

and $\eta < R$ if $2i < e^c - 2$. Induction on i gives $s_i = \mathsf{u}$ while $2i < e^c - 2$ and $s_i = \mathsf{v}$ once $2i > e^c - 2$. For $i = 2M + 4$, $2i = 4M + 8 > 3M + 3 = e^c - 2$, so $s_{2M+4} = \mathsf{v}$.

(d) Under pinned decoding every step reads a row $\Pi\, s\, \mathsf{a}$ with $s \in \{\mathsf{u}, \mathsf{v}\}$, which returns $\delta(s, \mathsf{a}) \in \{\mathsf{u}, \mathsf{v}\}$ by (a). Lemma 2 with $S = \{\mathsf{u}, \mathsf{v}\}$, $B = \{\mathsf{a}\}$ and $F(s, \mathsf{a}) = \rho(\Pi\, s\, \mathsf{a})$ gives equality with δ on every input.

With $e^c = 2N + 5$ instead, the same computation gives a decoder that passes both rows and is exact under unpinned decoding on every input of length at most N, yet wrong on a^{N+2} (`no_finite_check`); testing every input up to a given length does not suffice either. Conversely, if the context moves every logit by at most τ from the logit at the pinned row and every certified margin exceeds 2τ, unpinned decoding is exact on every input (`unpinned_exact_of_close`). In the construction, the head's write is such a perturbation, and it grows without bound.

Theorem 3

Definition 5 (Transcript). *A transcript makes K calls. Call i receives the input $x_i = q_i(v)$ and returns $v_i = E_i(x_i)$, where E_i is the procedure that executes it and q_i reads only $v_1, \ldots, v_{i-1}$; the answer $g(v)$ reads only $v_1, \ldots, v_K$. Call i has a table T_i, the function its procedure is certified to compute. The intended computation has inputs q_i^*, functions T_i^* and combining program g^*, with results satisfying $v_i^* = T_i^*(q_i^*(v^*))$ and answer $g^*(v^*)$.*

Theorem 3 (Composition, precise form). *(a) If $g(v) \neq g^*(v^*)$, then for some $i \leq K$, $E_i(x_i) \neq T_i(x_i)$, or $T_i(q_i^*(v^*)) \neq T_i^*(q_i^*(v^*))$, or $q_i(v^*) \neq q_i^*(v^*)$; or else $g(v^*) \neq g^*(v^*)$. (b) Let every quantity depend on a transcript ω in a set Ω with a measure μ. If $E_i(x_i) = T_i(x_i)$ for every call of every transcript, then*

$$\begin{aligned}\mu\{g(v) \neq g^*(v^*)\} \leq\ & \mu\{\exists i\colon T_i(q_i^*(v^*)) \neq T_i^*(q_i^*(v^*))\} \\ & + \mu\{\exists i\colon q_i(v^*) \neq q_i^*(v^*)\} + \mu\{g(v^*) \neq g^*(v^*)\}.\end{aligned}$$

Proof. (a) Suppose that none of the four events occurs. We show $v_i = v_i^*$ for all $i \leq K$ by strong induction on i. If $v_j = v_j^*$ for all $j < i$, then $q_i(v) = q_i(v^*)$ because q_i reads only earlier results, and

$$v_i = E_i(q_i(v)) = T_i(q_i(v)) = T_i(q_i(v^*)) = T_i(q_i^*(v^*)) = T_i^*(q_i^*(v^*)) = v_i^*,$$

using in turn that the procedure, the transcription and the table are correct. Then $g(v) = g(v^*) = g^*(v^*)$, because g reads only the results and the combining program is correct, a contradiction.

(b) Under the hypothesis, (a) places the event $\{g(v) \neq g^*(v^*)\}$ inside the union of the other three, and the bound follows from monotonicity and subadditivity of μ, which hold for arbitrary sets, so no measurability is needed; a probability distribution gives (5). For a procedure certified on the views it reaches, the hypothesis $E_i(x_i) = T_i(x_i)$ is Theorem 1 applied to its run on x_i, whatever the length of x_i and the order of the calls. In Lean, (a) is proved for arbitrary E_i, T_i and T_i^*, and (b) for calls executed by the token-form procedure, with the hypothesis derived from Theorem 1 in that form (`compose_exact`).

Theorem 4

Proof of Theorem 4. The first statement is Lemma 1. For the second, fix $c_0 \in I$ and let the execution path's step t be $G_t(c) = \mathrm{paint}(c, \tilde{f}_t(\mathrm{view}(c)))$ if $\mathrm{view}(c) \neq \bot$ and $G_t(c) = c$ otherwise, where $\tilde{f}_t(v)$ is the argmax of the execution-path logits $\tilde{z}_k^{(t)}(v)$ at each position; these may depend on t, and through it on the rest of the batch. Let $c \in \mathrm{Reach}_T(\{c_0\})$ with $v = \mathrm{view}(c) \neq \bot$. Then $v \in R_T(I) \subseteq V$, so $\|\tilde{z}_k^{(t)}(v) - z_k(v)\|_\infty \leq \tau$ and $m(v) > 2\tau$, and Lemma 1 at each position gives $\tilde{f}_t(v) = T(v)$, hence $G_t(c) = \sigma_T(c)$; this also holds when $\mathrm{view}(c) = \bot$. Lemma 3 with a single event, so that the history is the step count, gives $G_t \circ \cdots \circ G_1(c_0) = \sigma_T^t(c_0)$ for every t.

In Lean, the first statement is `sel_eq_of_close` and Lemma 3 is `histRun_eq_of_closed`; their combination is `regeneration_of_close` for the token form and `unpinned_exact_of_close` for unpinned decoding, and the sheet form is the instance above. By Lemma 4, a pinned step computed on a cache of a fixed prefix, or decoded token by token on it, yields exactly the logits of the full prompt (`cachedForward_eq`, `policy_tick_eq`), so in exact arithmetic it is an execution path with $\tau = 0$.

B Certificate format

A certificate is a JSON file distributed with the model graph. It specifies the prompt forms under which it was measured, the numerical precision of each graph, the library version, hashes of the glyph set and of every model file, the device and the time, and, for each table, the number of rows, the number passed and every row's margin in a fixed order, to 10^{-3} logits. A client accepts a certificate only if every hash matches the loaded files, and recomputes the lowest-margin tenth of each table before use.

C Reproducibility

Every number in this paper is listed, with its source record, in `paper/numbers.md` of the Sketchpad repository. Training, server certification, the gate identity test and dialogue fine-tuning are scripted for Modal; the device certificate and browser test run in the client; the algebra tests and cost tables run with `pnpm vitest run src/lib/sketchpad/program`; the comparison is in `paper/baselines/`; the Lean proofs build with `lake build`.

References

1. Wei, J. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In Advances in Neural Information Processing Systems, 24824–24837 (2022). https://arxiv.org/abs/2201.11903
2. Nye, M. et al. Show Your Work: Scratchpads for Intermediate Computation with Language Models. Preprint at arXiv:2112.00114 (2021). https://arxiv.org/abs/2112.00114
3. Nogueira, R., Jiang, Z. & Lin, J. Investigating the Limitations of Transformers with Simple Arithmetic Tasks. Preprint at arXiv:2102.13019 (2021). https://arxiv.org/abs/2102.13019
4. Anil, C. et al. Exploring Length Generalization in Large Language Models. In Advances in Neural Information Processing Systems, 38546–38556 (2022). https://arxiv.org/abs/2207.04901
5. Lee, N., Sreenivasan, K., Lee, J. D., Lee, K. & Papailiopoulos, D. Teaching Arithmetic to Small Transformers. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2307.03381
6. Zhou, H. et al. What Algorithms can Transformers Learn? A Study in Length Generalization. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2310.16028
7. Zhou, Y. et al. Transformers Can Achieve Length Generalization But Not Robustly. Preprint at arXiv:2402.09371 (2024). https://arxiv.org/abs/2402.09371

8. McLeish, S. et al. Transformers Can Do Arithmetic with the Right Embeddings. In Advances in Neural Information Processing Systems, 108012–108041 (2024). https://doi.org/10.52202/079017-3430
9. Cho, H. et al. Position Coupling: Improving Length Generalization of Arithmetic Transformers Using Task Structure. In Advances in Neural Information Processing Systems, 22233–22315 (2024). https://doi.org/10.52202/079017-0701
10. Schick, T. et al. Toolformer: Language Models Can Teach Themselves to Use Tools. In Advances in Neural Information Processing Systems, 68539–68551 (2023). https://doi.org/10.52202/075280-2997
11. Gao, L. et al. PAL: Program-aided Language Models. In Proceedings of the 40th International Conference on Machine Learning, 10764–10799 (2023). https://arxiv.org/abs/2211.10435
12. Chen, W., Ma, X., Wang, X. & Cohen, W. W. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. Transactions on Machine Learning Research (2023). https://arxiv.org/abs/2211.12588
13. Gou, Z. et al. ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2309.17452
14. Feng, J. et al. ReTool: Reinforcement Learning for Strategic Tool Use in LLMs. In International Conference on Learning Representations (ICLR 2026) (2026). https://arxiv.org/abs/2504.11536
15. Moore, E. F. Gedanken-Experiments on Sequential Machines. In Automata Studies, 129–153 (1956). https://doi.org/10.1515/9781400882618-006
16. Cai, J., Shin, R. & Song, D. Making Neural Programming Architectures Generalize via Recursion. In International Conference on Learning Representations (ICLR 2017) (2017). https://arxiv.org/abs/1704.06611
17. Lewandowski, A., Machado, M. C. & Schuurmans, D. Universal computation is intrinsic to language model decoding. Preprint at arXiv:2601.08061 (2026). https://arxiv.org/abs/2601.08061
18. Hu, E. J. et al. LoRA: Low-Rank Adaptation of Large Language Models. In International Conference on Learning Representations (ICLR 2022) (2022). https://arxiv.org/abs/2106.09685
19. Qwen Team. Qwen3.5: Towards Native Multimodal Agents. Qwen blog (2026). https://qwen.ai/blog?id=qwen3.5
20. Yang, S., Kautz, J. & Hatamizadeh, A. Gated Delta Networks: Improving Mamba2 with Delta Rule. In International Conference on Learning Representations (ICLR 2025) (2025). https://arxiv.org/abs/2412.06464
21. Li, Z. & Hoiem, D. Learning without Forgetting. IEEE Transactions on Pattern Analysis and Machine Intelligence **40**, 2935–2947 (2018). https://doi.org/10.1109/TPAMI.2017.2773081
22. ONNX Runtime developers. ONNX Runtime. Software. Source https://github.com/microsoft/onnxruntime, MIT license; citation per the repository CITATION.cff (date-released 29 November 2018); the demo uses onnxruntime-web 1.26.0-dev.20260416 through Transformers.js 4.2.0 (2018). https://onnxruntime.ai
23. Hugging Face. Transformers.js. GitHub repository. Software, version 4.2.0 (released 23 April 2026), Apache-2.0 (2026). https://github.com/huggingface/transformers.js
24. Ninomiya, K., Jones, B. & Blandy, J. WebGPU. W3C Candidate Recommendation Draft, 15 September 2026. Latest version: https://www.w3.org/TR/webgpu/ (2026). https://www.w3.org/TR/2026/CRD-webgpu-20260915/
25. Knuth, D. E. The Art of Computer Programming, Volume 2: Seminumerical Algorithms. Third edn, Addison-Wesley (1997).
26. Kaltofen, E. & Trager, B. M. Computing with Polynomials Given by Black Boxes for Their Evaluations: Greatest Common Divisors, Factorization, Separation of Numerators and Denominators. Journal of Symbolic Computation **9**, 301–320 (1990). https://doi.org/10.1016/S0747-7171(08)80015-6
27. Kronecker, L. Grundzüge einer arithmetischen Theorie der algebraischen Grössen. Journal für die reine und angewandte Mathematik **92**, 1–122 (1882). https://doi.org/10.1515/crll.1882.92.1
28. Zhou, J. et al. Instruction-Following Evaluation for Large Language Models. Preprint at arXiv:2311.07911 (2023). https://arxiv.org/abs/2311.07911
29. McCloskey, M. & Cohen, N. J. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In Psychology of Learning and Motivation, 109–165 (1989). https://doi.org/10.1016/S0079-7421(08)60536-8
30. Kirkpatrick, J. et al. Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences **114**, 3521–3526 (2017). https://doi.org/10.1073/pnas.1611835114
31. Agarwal, R. et al. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2306.13649
32. Lu, K. & Thinking Machines Lab. On-Policy Distillation. Thinking Machines Lab: Connectionism. Blog post, 27 October 2025 (2025). https://doi.org/10.64434/tml.20251026
33. Shenfeld, I., Pari, J. & Agrawal, P. RL's Razor: Why Online Reinforcement Learning Forgets Less. In International Conference on Learning Representations (ICLR 2026) (2026). https://arxiv.org/abs/2509.04259
34. Shao, Z. et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. Preprint at arXiv:2402.03300 (2024). https://arxiv.org/abs/2402.03300
35. Cobbe, K. et al. Training Verifiers to Solve Math Word Problems. Preprint at arXiv:2110.14168 (2021). https://arxiv.org/abs/2110.14168
36. Hendrycks, D. et al. Measuring Massive Multitask Language Understanding. In International Conference on

Learning Representations (ICLR 2021) (2021). https://arxiv.org/abs/2009.03300
37. nostalgebraist. interpreting GPT: the logit lens. LessWrong. Blog post, 31 August 2020 (2020). https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens
38. Belrose, N. et al. Eliciting Latent Predictions from Transformers with the Tuned Lens. Preprint at arXiv:2303.08112 (2023). https://arxiv.org/abs/2303.08112
39. Turner, A. M. et al. Steering Language Models With Activation Engineering. Preprint at arXiv:2308.10248 (2023). https://arxiv.org/abs/2308.10248
40. Rimsky, N. et al. Steering Llama 2 via Contrastive Activation Addition. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 15504–15522 (2024). https://doi.org/10.18653/v1/2024.acl-long.828
41. Kaiser, Ł. & Sutskever, I. Neural GPUs Learn Algorithms. In International Conference on Learning Representations (ICLR 2016) (2016). https://arxiv.org/abs/1511.08228
42. Price, E., Zaremba, W. & Sutskever, I. Extensions and Limitations of the Neural GPU. Preprint at arXiv:1611.00736 (2016). https://arxiv.org/abs/1611.00736
43. Trask, A. et al. Neural Arithmetic Logic Units. In Advances in Neural Information Processing Systems (2018). https://arxiv.org/abs/1808.00508
44. Madsen, A. & Johansen, A. R. Neural Arithmetic Units. In International Conference on Learning Representations (ICLR 2020) (2020). https://arxiv.org/abs/2001.05016
45. Liu, T. & Low, B. K. H. Goat: Fine-tuned LLaMA Outperforms GPT-4 on Arithmetic Tasks. Preprint at arXiv:2305.14201 (2023). https://arxiv.org/abs/2305.14201
46. Yang, Z. et al. GPT Can Solve Mathematical Problems Without a Calculator. Preprint at arXiv:2309.03241 (2023). https://arxiv.org/abs/2309.03241
47. Shen, S., Shen, P. & Zhu, D. RevOrder: A Novel Method for Enhanced Arithmetic in Language Models. Preprint at arXiv:2402.03822 (2024). https://arxiv.org/abs/2402.03822
48. Charton, F. Learning the greatest common divisor: explaining transformer predictions. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2308.15594
49. Kasliwal, S. Algorithmic Scratchpads and Curriculum Staging for Arithmetic Reasoning in Tiny Transformers. Preprint at arXiv:2610.09003 (2026). https://arxiv.org/abs/2610.09003
50. Wei, Z. On the Mirage of Long-Range Dependency, with an Application to Integer Multiplication. Preprint at arXiv:2603.29069 (2026). https://arxiv.org/abs/2603.29069
51. Wei, Z. On the Spatiotemporal Dynamics of Generalization in Neural Networks. Preprint at arXiv:2602.01651 (2026). https://arxiv.org/abs/2602.01651
52. Papazov, H., D'Angelo, F. & Flammarion, N. Exact Learning of Arithmetic with Differentiable Agents. Preprint at arXiv:2511.22751 (2025). https://arxiv.org/abs/2511.22751
53. Quirke, P., Neo, C. & Barez, F. Understanding Addition and Subtraction in Transformers. Preprint at arXiv:2402.02619 (2024). https://arxiv.org/abs/2402.02619
54. Back de Luca, A., Giapitzakis, G. & Fountoulakis, K. Learning to Add, Multiply, and Execute Algorithmic Instructions Exactly with Neural Networks. In Advances in Neural Information Processing Systems, 87469–87513 (2025). https://doi.org/10.52202/085713-2636
55. Lai, J. et al. Executing Arithmetic: Fine-Tuning Large Language Models as Turing Machines. Preprint at arXiv:2410.07896 (2024). https://arxiv.org/abs/2410.07896
56. Dugan, O. et al. OccamLLM: Fast and Exact Language Model Arithmetic in a Single Step. In Advances in Neural Information Processing Systems, 35665–35699 (2024). https://doi.org/10.52202/079017-1125
57. Dietz, F. & Klakow, D. IGC: Integrating a Gated Calculator into an LLM to Solve Arithmetic Tasks Reliably and Efficiently. Preprint at arXiv:2501.00684 (2025). https://arxiv.org/abs/2501.00684
58. Giannou, A. et al. Looped Transformers as Programmable Computers. In Proceedings of the 40th International Conference on Machine Learning, 11398–11442 (2023). https://arxiv.org/abs/2301.13196
59. Turkcan, M. K. Loom: A Scalable Analytical Neural Computer Architecture. Preprint at arXiv:2604.08816 (2026). https://arxiv.org/abs/2604.08816
60. Percepta. transformer-vm. GitHub repository. Software (Apache-2.0): compile programs directly into transformer weights; repository created 25 March 2026; accessed 31 October 2026 (2026). https://github.com/Percepta-Core/transformer-vm
61. Weng, Y. et al. Mastering Symbolic Operations: Augmenting Language Models with Compiled Neural Networks. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2304.01665
62. Xu, R., Yang, C., Li, Y. & McAllester, D. Towards Training Transformers as a Universal Computer. In Findings of the Association for Computational Linguistics: EMNLP 2026, 48933–48954 (2026). https://doi.org/10.18653/2026.findings-emnlp.2284
63. Li, C. et al. Chain of Code: Reasoning with a Language Model-Augmented Code Emulator. In Proceedings of the 41st International Conference on Machine Learning, 28259–28277 (2024). https://arxiv.org/abs/2312.04474
64. Qian, C. et al. ToolRL: Reward is All Tool Learning Needs. In Advances in Neural Information Processing Systems, 116896–116926 (2025). https://doi.org/10.52202/085713-3524

65. Zhang, A. L., Kraska, T. & Khattab, O. Recursive Language Models. Preprint at arXiv:2512.24601 (2025). https://arxiv.org/abs/2512.24601
66. Katz, G., Barrett, C., Dill, D. L., Julian, K. & Kochenderfer, M. J. Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks. In Computer Aided Verification (CAV 2017), 97–117 (2017). https://doi.org/10.1007/978-3-319-63387-9_5
67. Chow, T. S. Testing Software Design Modeled by Finite-State Machines. IEEE Transactions on Software Engineering **SE-4**, 178–187 (1978). https://doi.org/10.1109/TSE.1978.231496
68. Schuurmans, D., Dai, H. & Zanini, F. Autoregressive Large Language Models are Computationally Universal. Preprint at arXiv:2410.03170 (2024). https://arxiv.org/abs/2410.03170
69. Abbe, E., Bengio, S., Lotfi, A., Sandon, C. & Saremi, O. How Far Can Transformers Reason? The Globality Barrier and Inductive Scratchpad. In Advances in Neural Information Processing Systems, 27850–27895 (2024). https://doi.org/10.52202/079017-0874
70. Yang, C., Srebro, N., McAllester, D. & Li, Z. PENCIL: Long Thoughts with Short Memory. In Proceedings of the 42nd International Conference on Machine Learning, 71092–71156 (2025). https://arxiv.org/abs/2503.14337
71. Back de Luca, A. & Fountoulakis, K. Certification from Examples is Hard for Circuits and Transformers under Minimal Overparametrization. Preprint at arXiv:2605.22964 (2026). https://arxiv.org/abs/2605.22964
72. Yang, A., Bergsträßer, P., Zetzsche, G., Chiang, D. & Lin, A. W. Length Generalization Bounds for Transformers. In Proceedings of the 43rd International Conference on Machine Learning, 144227–144248 (2026). https://arxiv.org/abs/2603.02238
73. Rust, P. et al. Language Modelling with Pixels. In International Conference on Learning Representations (ICLR 2023) (2023). https://arxiv.org/abs/2207.06991
74. Tschannen, M., Mustafa, B. & Houlsby, N. CLIPPO: Image-and-Language Understanding from Pixels Only. In 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 11006–11017 (2023). https://doi.org/10.1109/CVPR52729.2023.01059
75. Lee, K. et al. Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. In Proceedings of the 40th International Conference on Machine Learning, 18893–18912 (2023). https://arxiv.org/abs/2210.03347
76. Wei, H., Sun, Y. & Li, Y. DeepSeek-OCR: Contexts Optical Compression. Preprint at arXiv:2510.18234 (2025). https://arxiv.org/abs/2510.18234
77. Cheng, J. et al. Glyph: Scaling Context Windows via Visual-Text Compression. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 37145–37158 (2026). https://doi.org/10.18653/v1/2026.acl-long.1722
78. Balter, S. G., Jerzak, E. & Jerzak, C. T. Multiplication in Multimodal LLMs: Computation with Text, Image, and Audio Inputs. In Findings of the Association for Computational Linguistics: ACL 2026, 40766–40780 (2026). https://doi.org/10.18653/v1/2026.findings-acl.2025
79. Hu, Y. et al. Visual Sketchpad: Sketching as a Visual Chain of Thought for Multimodal Language Models. In Advances in Neural Information Processing Systems, 139348–139379 (2024). https://doi.org/10.52202/079017-4423
80. Menon, S., Zemel, R. & Vondrick, C. Whiteboard-of-Thought: Thinking Step-by-Step Across Modalities. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 20016–20031 (2024). https://doi.org/10.18653/v1/2024.emnlp-main.1117
81. Li, C. et al. Imagine While Reasoning in Space: Multimodal Visualization-of-Thought. In Proceedings of the 42nd International Conference on Machine Learning, 36340–36364 (2025). https://arxiv.org/abs/2501.07542
82. Akter, S. N., Madaan, A., Lee, S., Yang, Y. & Nyberg, E. Self-Imagine: Effective Unimodal Reasoning with Multimodal Models using Self-Imagination. Preprint at arXiv:2401.08025 (2024). https://arxiv.org/abs/2401.08025
83. Shi, W. et al. MathCanvas: Intrinsic Visual Chain-of-Thought for Multimodal Mathematical Reasoning. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 27933–27954 (2026). https://doi.org/10.18653/v1/2026.acl-long.1287
84. Sutherland, I. E. Sketchpad: A Man-Machine Graphical Communication System. PhD thesis, Massachusetts Institute of Technology (1963). http://hdl.handle.net/1721.1/14979
85. Lample, G. & Charton, F. Deep Learning for Symbolic Mathematics. In International Conference on Learning Representations (ICLR 2020) (2020). https://arxiv.org/abs/1912.01412
86. Agarwal, V., Aditya, S. & Goyal, N. Analyzing the Nuances of Transformers' Polynomial Simplification Abilities. Preprint at arXiv:2104.14095 (2021). https://arxiv.org/abs/2104.14095
87. Kera, H., Ishihara, Y., Kambe, Y., Vaccon, T. & Yokoyama, K. Learning to compute Gröbner bases. In Advances in Neural Information Processing Systems, 33141–33187 (2024). https://doi.org/10.52202/079017-1044
88. Lewkowycz, A. et al. Solving Quantitative Reasoning Problems with Language Models. In Advances in Neural Information Processing Systems, 3843–3857 (2022). https://doi.org/10.52202/068431-0278
89. Feng, G. et al. Towards Revealing the Mystery behind Chain of Thought: A Theoretical Perspective. In Advances in Neural Information Processing Systems, 70757–70798 (2023). https://doi.org/10.52202/075280-3100
90. Rusu, A. A. et al. Progressive Neural Networks. Preprint at arXiv:1606.04671 (2016). https://arxiv.org/abs/1606.04671

91. Buehler, E. L. & Buehler, M. J. X-LoRA: Mixture of low-rank adapter experts, a flexible framework for large language models with applications in protein mechanics and molecular design. APL Machine Learning **2** (2024). https://doi.org/10.1063/5.0203126
92. Rajani, N. et al. No Robots. Hugging Face dataset. Dataset HuggingFaceH4/no_robots, CC BY-NC 4.0 (2023). https://huggingface.co/datasets/HuggingFaceH4/no_robots
93. de Moura, L. & Ullrich, S. The Lean 4 Theorem Prover and Programming Language. In Automated Deduction – CADE 28, 625–635 (2021). https://doi.org/10.1007/978-3-030-79876-5_37
94. The mathlib Community. The Lean Mathematical Library. In Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), 367–381 (2020). https://doi.org/10.1145/3372885.3373824